

**PROPOSAL PROJECT AKHIR
PRAKTEK MIKROKONTROLER
FLOOD EARLY WARNING SYSTEM**



Disusun oleh :

Muhammad Athaozih (23090620043)
Ramdhan Javas D. (23090620048)
Nastiti Dhea Imtinan (23090620050)
Dyah Ayu Ramadhani (23090620062)
Ahmad Ilman Nadziron (23090620065)

**D4 TEKNIK ELEKTRONIKA
DEPARTEMEN ELEKTRO DAN ELEKTRONIKA
FAKULTAS VOKASI
UNIVERSITAS NEGERI YOGYAKARTA
2025**

BAB I

Pendahuluan

A. Latar Belakang

Banjir merupakan salah satu bencana alam yang paling sering terjadi di Indonesia dan menimbulkan dampak negatif yang signifikan terhadap aktivitas manusia, termasuk kemacetan lalu lintas, kerusakan infrastruktur, dan bahkan korban jiwa. Faktor penyebab banjir antara lain tingginya intensitas curah hujan, buruknya sistem drainase, serta berkurangnya daerah resapan air akibat urbanisasi dan pencemaran saluran air.

Data dari Badan Nasional Penanggulangan Bencana (BNPB) menunjukkan bahwa banjir merupakan bencana paling sering terjadi di Indonesia dalam beberapa tahun terakhir. Pada tahun 2023 saja, tercatat lebih dari 1.200 kejadian banjir di berbagai daerah, dengan Jawa Barat, Jakarta, Jawa Tengah, dan Kalimantan Selatan sebagai wilayah yang paling sering terdampak. Di Jakarta, banjir besar hampir terjadi setiap tahun, terutama saat musim hujan dengan curah hujan ekstrem dan kondisi drainase yang tidak memadai.

Kerugian akibat banjir juga sangat besar. Menurut catatan BNPB, total kerugian akibat banjir di Indonesia bisa mencapai triliunan rupiah per tahun, mencakup kerusakan rumah, fasilitas umum, infrastruktur jalan, dan kehilangan mata pencaharian masyarakat. Sebagai contoh, banjir besar yang melanda Jakarta dan sekitarnya pada awal tahun 2020 menyebabkan kerugian sekitar Rp1 triliun dan menewaskan puluhan orang.

Selain dampak ekonomi dan infrastruktur, banjir juga berdampak besar pada sektor kesehatan masyarakat. Kondisi genangan air dan lingkungan yang tidak higienis memicu munculnya berbagai penyakit, seperti leptospirosis, diare, infeksi saluran pernapasan akut (ISPA), dan penyakit kulit. Kasus leptospirosis, yang ditularkan melalui air yang terkontaminasi urine hewan, meningkat signifikan di daerah-daerah terdampak banjir.

Kondisi ini menuntut adanya upaya mitigasi yang efektif, salah satunya melalui pengembangan sistem peringatan dini banjir yang dapat memberikan informasi secara cepat dan akurat kepada masyarakat dan pihak terkait. Sistem ini penting untuk menginformasikan potensi banjir sebelum terjadi sehingga masyarakat dapat mengambil langkah antisipasi yang tepat dan mengurangi kerugian materiil maupun korban jiwa.

Sistem peringatan dini banjir biasanya memanfaatkan sensor untuk memantau parameter kritis seperti ketinggian muka air sungai dan intensitas

curah hujan. Data tersebut kemudian diolah dan disampaikan kepada masyarakat melalui berbagai media komunikasi seperti SMS, aplikasi mobile, atau media sosial. Contohnya, penggunaan water level sensor untuk mengukur ketinggian air dan sensor curah hujan yang terintegrasi dengan perangkat mikrokontroler seperti Arduino atau Raspberry Pi, yang memungkinkan pemantauan secara real-time dan otomatis.

Selain itu, sistem peringatan dini yang efektif juga membutuhkan model prediksi cuaca dan hidrologi berbasis teknologi seperti Numerical Weather Prediction (NWP) dan pemodelan hidrologi untuk memberikan prakiraan banjir dengan jangka waktu yang cukup agar badan penanggulangan bencana dapat merespons dengan cepat.

Dengan adanya sistem peringatan dini banjir yang handal, diharapkan dapat meminimalkan dampak buruk banjir, meningkatkan kesiapsiagaan masyarakat, dan mendukung koordinasi antar lembaga dalam penanggulangan bencana. Oleh karena itu, pengembangan dan implementasi Flood Early Warning System menjadi sangat penting sebagai bagian dari strategi pengurangan risiko bencana di Indonesia.

B. Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam pengembangan Sistem Peringatan Dini Banjir ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun sistem peringatan dini banjir yang mampu memberikan informasi secara tepat waktu dan akurat?
2. Bagaimana memanfaatkan teknologi sensor, seperti water level sensor, untuk deteksi banjir secara real time?
3. Bagaimana memanfaatkan teknologi sensor, seperti water level sensor, untuk melakukan deteksi potensi banjir secara real-time?
4. Bagaimana mengintegrasikan media komunikasi berbasis internet, seperti layanan Telegram, untuk menyampaikan peringatan dini kepada masyarakat secara cepat dan luas?
5. Bagaimana menguji keandalan dan efektivitas sistem peringatan dini banjir yang telah dirancang, baik dari segi akurasi pengukuran sensor, waktu respons sistem, maupun keberhasilan pengiriman notifikasi melalui Telegram kepada pengguna?

C. Batasan Masalah

Adapun batasan masalah dalam penelitian ini sebagai berikut :

1. Sistem peringatan dini banjir yang dirancang hanya difokuskan pada monitoring lokal secara real-time menggunakan water level sensor untuk mendeteksi ketinggian muka air atau saluran, tanpa

mempertimbangkan parameter hidrologis lainnya seperti curah hujan, debit sungai, atau kecepatan arus air.

2. Pengolahan data sensor dan pengendalian sistem peringatan dilakukan sepenuhnya menggunakan mikrokontroler ESP32, tanpa melibatkan sistem komputasi tambahan seperti Raspberry Pi, server cloud, ataupun perangkat pemrosesan eksternal lainnya.
3. Sistem peringatan yang dibangun hanya menggunakan media komunikasi berbasis internet, yaitu pengiriman notifikasi secara otomatis melalui aplikasi Telegram, tanpa integrasi dengan aplikasi web, mobile, atau media komunikasi lain.
4. Pengujian sistem dilakukan secara terkontrol melalui simulasi dan praktikum di lingkungan laboratorium, bukan di wilayah rawan banjir secara langsung. Evaluasi difokuskan pada aspek akurasi pembacaan sensor, kecepatan respons sistem terhadap perubahan ketinggian air, serta keberhasilan pengiriman peringatan melalui Telegram kepada pengguna.

BAB II

Tinjaun Pustaka

A. Deskripsi Sistem Secara Umum

Sistem peringatan dini banjir ini dirancang untuk memberikan informasi peringatan dini secara otomatis kepada masyarakat dan pihak terkait jika terjadi potensi banjir di suatu wilayah. sistem ini bekerja dengan memantau ketinggian air secara real-time menggunakan water level sensor yang ditempatkan di saluran air. data dari sensor diolah oleh mikrokontroler, dan jika ketinggian air mencapai level tertentu yang telah ditentukan, sistem akan mengirimkan peringatan melalui berbagai media komunikasi.

Alur kerja sistem secara umum:

1. Deteksi Tinggi Air : Water Level Sensor

Sensor ketinggian air water level sensor dipasang pada dinding sungai atau saluran air untuk memantau permukaan air secara terus-menerus. Sensor ini menghasilkan sinyal analog berupa tegangan atau pulsa pantulan ultrasonik yang merepresentasikan ketinggian muka air. Setiap selang waktu tertentu, biasanya setiap 1–2 detik, sensor akan mengirimkan data ke pin analog atau digital pada ESP32 untuk kemudian di-sampling dan diproses lebih lanjut.

2. Mikrokontroler : ESP32

Mikrokontroler ESP32 membaca data tinggi muka air dari sensor menggunakan ADC 12-bit untuk sensor analog atau menghitung waktu pantulan jika menggunakan sensor ultrasonik. Nilai ketinggian tersebut dibandingkan dengan tiga ambang yang telah ditentukan, yaitu Normal ($< 60\%$ kapasitas), Siaga ($60\text{--}90\%$), dan Bahaya ($\geq 90\%$). Berdasarkan logika program, ESP32 akan memicu LED dan buzzer sebagai alarm lokal, menyiapkan pesan peringatan untuk dikirimkan melalui layanan Telegram, serta mencatat waktu serta status ke dalam memori EEPROM atau kartu SD.

3. Pengiriman Peringatan : Telegram Bot melalui Wi-Fi

Untuk pengiriman peringatan jarak jauh, ESP32 menggunakan koneksi Wi-Fi untuk mengakses internet dan mengirimkan pesan melalui Telegram Bot API. Pesan yang dikirim berisi informasi status banjir, lokasi, waktu, dan tinggi muka air kepada akun Telegram atau grup tertentu yang telah didaftarkan. Sistem juga dapat mengatur pengiriman ulang (retry) jika koneksi internet terputus sementara, dan selama gangguan tersebut, data disimpan secara lokal hingga koneksi kembali tersedia.

4. Indikator Lokal : Buzzer dan LED

Sistem juga dilengkapi indikator visual dan audio berupa LED dan buzzer. Ketika status berada pada level Siaga, LED kuning menyala statis dan buzzer berbunyi bip pendek setiap 5 detik. Jika status meningkat menjadi Bahaya, LED merah berkedip dan buzzer menyala

secara terus-menerus atau dalam pola intermiten cepat. Saat kondisi kembali Normal, LED hijau menyala dan buzzer otomatis mati.

5. Catu Daya : Adaptor

Sistem memperoleh daya utama dari adaptor DC 12V–2 A. Tegangan ini kemudian diturunkan menjadi 5 V menggunakan modul step-down (buck converter) untuk mendukung ESP32, sensor, LED, dan komponen lainnya. Selain itu, sistem dilengkapi dengan baterai cadangan Li-ion 18650 (1–2 sel) yang terhubung dengan modul charger TP4056, berfungsi sebagai sistem UPS mikro yang menjaga sistem tetap aktif ketika sumber utama terputus.

6. Siklus Berulang & Penyimpanan Data

Sistem bekerja dalam loop yang berulang setiap beberapa detik. Pada setiap siklus, ESP32 akan membaca data sensor, memproses logika bahaya, mengaktifkan peringatan bila perlu, serta mengirim peringatan jika terjadi perubahan status. Semua peristiwa penting seperti waktu kejadian, level air, status pengiriman pesan Telegram, dan tegangan baterai dicatat dalam memori EEPROM atau SD card untuk kepentingan pemantauan dan evaluasi kinerja sistem.

B. Pemilihan Komponen

1. Water Level Sensor

Sensor ini digunakan untuk memantau ketinggian air di saluran secara real-time. Data yang dihasilkan akan diolah oleh Arduino untuk menentukan apakah ketinggian air telah mencapai level bahaya banjir.

- Spesifikasi : Memiliki ukuran 59 mm x 20 mm, sensor ini mudah dipasang di saluran air tanpa memakan banyak ruang, lalu memungkinkan pengukuran ketinggian air yang lebih presisi dibandingkan sensor digital on/off, sehingga sistem dapat mendeteksi kenaikan air secara bertahap.
- Alasan Pemilihan :
 - Sensor ini ideal untuk mendeteksi ketinggian air secara langsung dengan metode kontak. Output analognya memungkinkan Arduino membaca perubahan ketinggian air secara bertahap, yang penting untuk sistem peringatan dini banjir yang cocok untuk memicu peringatan ketika air mencapai level tertentu yang telah ditentukan.
 - Sensor ini mudah dihubungkan ke Arduino melalui pin analog, dan tidak memerlukan konversi sinyal tambahan.
 - Sensor yang bersifat kontak dengan air, desainnya cukup sederhana untuk digunakan di saluran air tawar.
 - Memiliki konsumsi daya yang rendah, sehingga tidak membebani sumber daya sistem (misalnya adapter atau baterai).

2. ESP32

Berfungsi sebagai mikrokontroler utama yang mengolah data dari sensor ketinggian air, membuat keputusan berdasarkan logika yang diprogram, dan mengendalikan pengiriman peringatan melalui layanan Telegram berbasis internet.

- Spesifikasi : Mikrokontroler berbasis dual-core Tensilica Xtensa LX6, memiliki GPIO yang banyak untuk koneksi sensor dan modul, mendukung pemrograman via USB dan konektivitas Wi-Fi/Bluetooth bawaan, serta efisiensi daya yang baik.
- Alasan Pemilihan :
 - ESP32 mampu membaca data sensor, memprosesnya, dan mengirimkan peringatan melalui internet secara efisien tanpa memerlukan modul tambahan seperti GSM
 - ESP32 memiliki jumlah pin dan fitur yang cukup untuk menghubungkan sensor, LED, buzzer, serta mendukung komunikasi nirkabel. Selain itu, komunitas pengembang ESP32 juga sangat aktif, memudahkan dalam proses pengembangan dan troubleshooting.
 - Dengan harga yang tetap terjangkau, dukungan konektivitas internet, dan kemampuan pemrosesan yang lebih tinggi, ESP32 sangat cocok digunakan dalam sistem peringatan dini banjir yang membutuhkan pengiriman informasi cepat dan real-time.

3. Kabel USB

Kabel USB digunakan untuk menghubungkan ESP-32 ke komputer untuk keperluan pemrograman, debugging, atau sebagai sumber daya sementara selama pengembangan.

- Spesifikasi: Kabel USB tipe A ke tipe B (atau micro USB, tergantung model Arduino) dengan panjang yang memadai (biasanya 1-2 meter) dan mendukung transfer data serta daya.
- Alasan Pemilihan :
 - Dapat mengunggah kode program ke Arduino dan komunikasi serial untuk pengujian.
 - Kabel USB standar kompatibel dengan sebagian besar model Arduino (seperti Arduino Uno) dan mudah didapat.
 - Kabel ini diperlukan selama tahap pengembangan dan pengujian sistem walaupun nantinya diganti oleh adapter.

4. Buzzer

Buzzer digunakan sebagai indikator audio yang berbunyi saat sistem memasuki status Siaga atau Bahaya, sehingga masyarakat memperoleh peringatan langsung dan jelas.

- Spesifikasi: Buzzer yang digunakan adalah tipe piezoelectric aktif dengan tegangan kerja 3 – 5,5 V DC dan arus sekitar 30 mA pada 5 V. Perangkat ini menghasilkan suara ≥ 85 dB pada jarak 30 cm dengan frekuensi resonansi 2 – 4 kHz. Dimensinya ringkas (diameter ± 12 mm, tinggi ± 9 mm) dan hanya memiliki dua pin (VCC dan GND), sehingga mudah diintegrasikan langsung ke pin digital Arduino.
- Alasan Pemilihan:
 - Suara keras dan mudah dikenali bahkan di lingkungan bising.
 - Konsumsi daya rendah; dapat disuplai langsung dari Arduino tanpa driver tambahan.
 - Ukuran kecil dan pemasangan sederhana.
 - Waktu respons instan, cocok untuk alarm darurat.

5. LED

LED berfungsi sebagai indikator visual status sistem: hijau (AMAN), kuning (SIAGA), dan merah (BAHAYA).

- Spesifikasi: LED super-bright berdiameter 5 mm digunakan untuk memastikan visibilitas tinggi. Tegangan maju LED merah dan kuning berkisar 2,0 – 2,2 V, sedangkan LED hijau 3,0 – 3,2 V, dengan arus operasi 10 – 20 mA melalui resistor pembatas 220 – 330 Ω . Intensitas cahaya mencapai ± 3000 mcd (merah/hijau) dan ± 1500 mcd (kuning), dengan sudut pancar sekitar 30°.
- Alasan Pemilihan:
 - Indikator visual terang dan mudah dilihat dari jarak jauh.
 - Hemat daya dan umur pakai panjang (> 50.000 jam).
 - Pilihan warna jelas, memudahkan klasifikasi status sistem.

6. LCD 16x2 + I2C Module

Berfungsi untuk menampilkan informasi secara real-time seperti ketinggian air, status level air (Normal, Siaga, Bahaya), dan status sistem secara keseluruhan, sehingga pengguna dapat langsung mengetahui kondisi tanpa perlu membuka aplikasi.

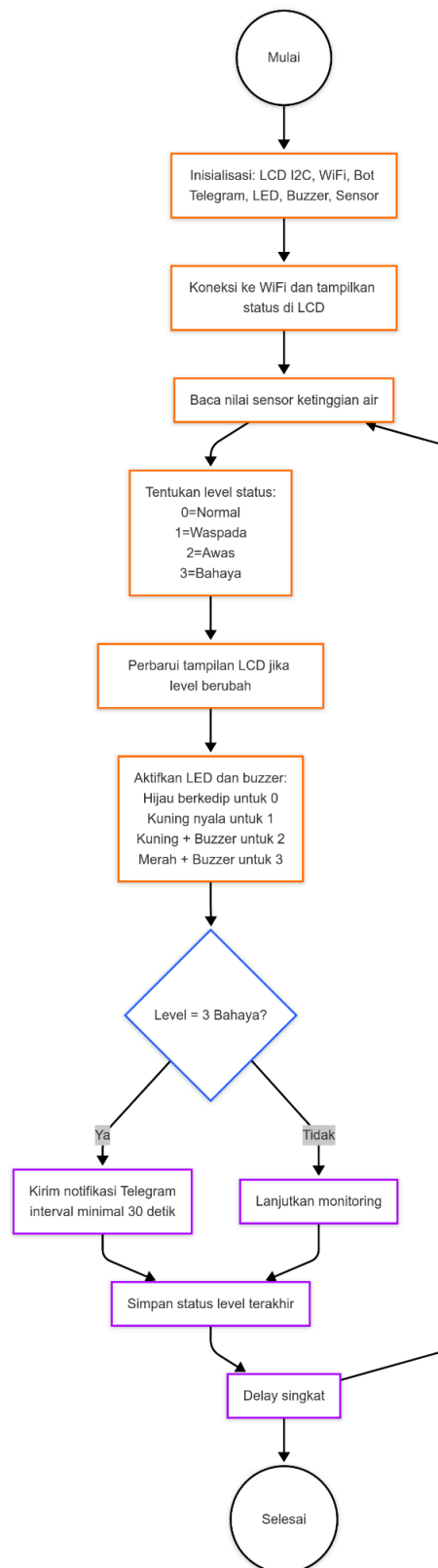
- Spesifikasi: Layar LCD karakter 16 kolom $\times$ 2 baris dengan antarmuka I2C, menggunakan chip driver seperti PCF8574. Modul I2C memungkinkan penghematan pin GPIO karena hanya menggunakan dua pin (SDA dan SCL), serta dilengkapi potensiometer untuk mengatur kontras tampilan.
- Alasan Pemilihan:

- Modul LCD 16x2 dengan I2C sangat hemat pin, cocok digunakan pada sistem berbasis ESP32 yang memiliki banyak fungsi lain.
- Layar ini cukup untuk menampilkan data penting secara ringkas dan jelas di lapangan.
- Komponen ini banyak tersedia, mudah digunakan dengan pustaka (library) seperti LiquidCrystal_I2C, dan sangat cocok untuk sistem monitoring sederhana dengan visibilitas langsung.

BAB III

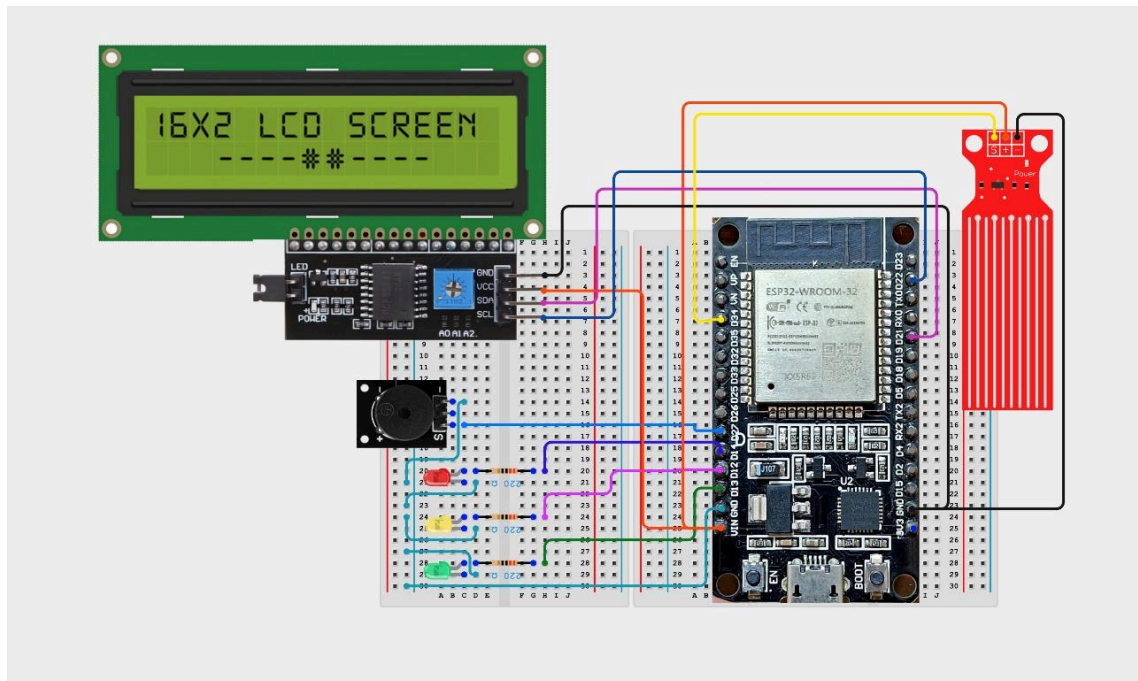
Metodologi

A. Diagram Alir Sistem



Gambar 1. Diagram Alir Sistem

B. Diagram Skematik



Gambar 2. Diagram Skematik.

C. Perancangan Pengujian

Sistem Flood Early Warning System (FEWS) ini dirancang untuk memberikan peringatan dini terhadap potensi banjir dengan memanfaatkan mikrokontroler ESP32, sensor ketinggian air, dan pengiriman notifikasi melalui platform Telegram. Sensor ketinggian air dipasang pada saluran atau tangki simulasi untuk mendeteksi perubahan muka air secara real-time. Data dari sensor dikirimkan ke ESP32 yang bertugas memproses sinyal dan menentukan tingkat kewaspadaan berdasarkan ambang batas yang telah ditentukan, yaitu status NORMAL, WASPADA, AWAS, dan BAHAYA. Informasi status ini divisualisasikan melalui LCD 16x2 serta direspons secara lokal dengan indikator LED berwarna dan buzzer sebagai alarm. Masing-masing status memiliki karakteristik visual dan audio yang berbeda, mulai dari nyala LED hijau tanpa suara untuk status NORMAL hingga LED merah dengan buzzer menyala terus menerus untuk status BAHAYA. Notifikasi secara daring dikirimkan secara otomatis ke akun Telegram pengguna agar informasi dapat diterima dengan cepat dan tepat.

Seluruh sistem ditenagai melalui adaptor 5V USB dan dapat dilengkapi dengan baterai cadangan untuk memastikan sistem tetap aktif dalam kondisi darurat. Pengujian sistem dilakukan secara bertahap dalam lingkungan simulasi yang merepresentasikan kondisi nyata di lapangan. Pada tahap awal, air dijaga dalam posisi rendah untuk memastikan status tetap NORMAL, dimana tidak ada notifikasi yang dikirim dan hanya LED hijau yang menyala.

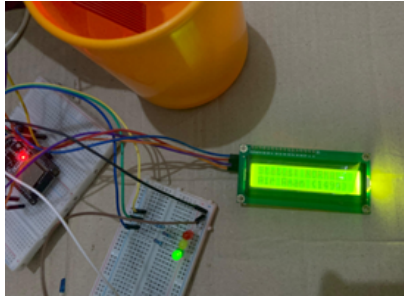
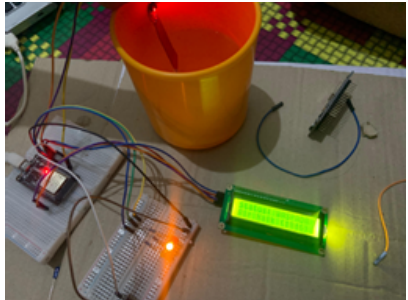
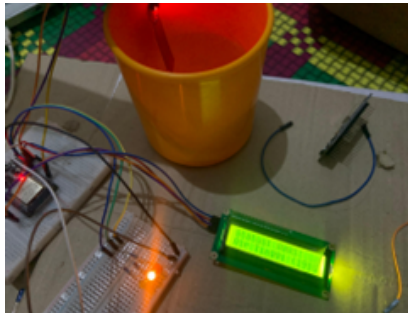
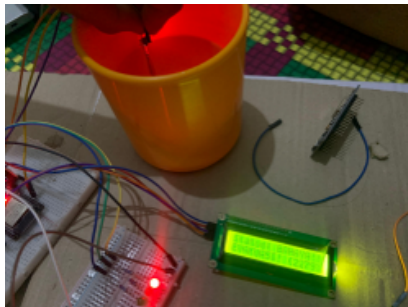
Ketika air dinaikkan hingga mencapai sekitar 60% dari kapasitas tangki, status berubah menjadi WASPADA dan sistem secara otomatis menyalakan LED kuning, membunyikan buzzer secara intermiten, serta mengirimkan peringatan ke Telegram. Kenaikan lebih lanjut hingga 80% menyebabkan sistem beralih ke status AWAS dengan LED oranye menyala dan buzzer berbunyi lebih cepat, serta notifikasi tambahan dikirimkan. Jika permukaan air melebihi ambang 90%, status berubah menjadi BAHAYA, LED merah menyala, buzzer berbunyi terus menerus tanpa jeda, dan sistem mengirimkan pesan peringatan kritis berulang kali melalui Telegram.

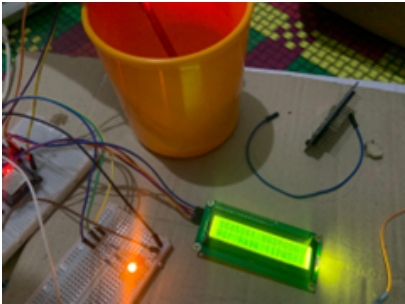
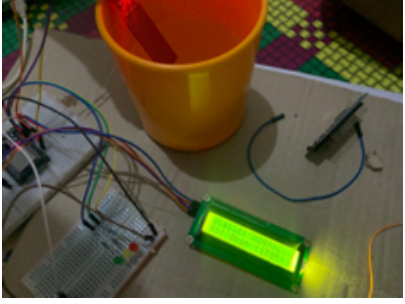
Selama pengujian, dilakukan pula simulasi kegagalan komunikasi dengan memutus jaringan Wi-Fi selama sepuluh menit. Sistem berhasil menyimpan data secara lokal dan mengirimkan kembali notifikasi yang tertunda setelah koneksi kembali aktif. Selain itu, saat sensor air dilepas dari sistem, ESP32 mampu mengenali gangguan tersebut sebagai kesalahan, mengalihkan status ke mode aman, dan mengirimkan pesan pemeliharaan ke Telegram. Setelah permukaan air kembali turun di bawah ambang batas WASPADA, sistem secara otomatis menonaktifkan alarm, mengubah status menjadi NORMAL, dan mencatat waktu transisi. Evaluasi dilakukan terhadap kecepatan respons, akurasi pembacaan sensor, keandalan pengiriman notifikasi, dan kestabilan daya untuk memastikan sistem ini layak diimplementasikan sebagai bagian dari solusi mitigasi risiko banjir berbasis teknologi.

BAB IV

Hasil dan Pembahasan

A. Hasil Implementasi

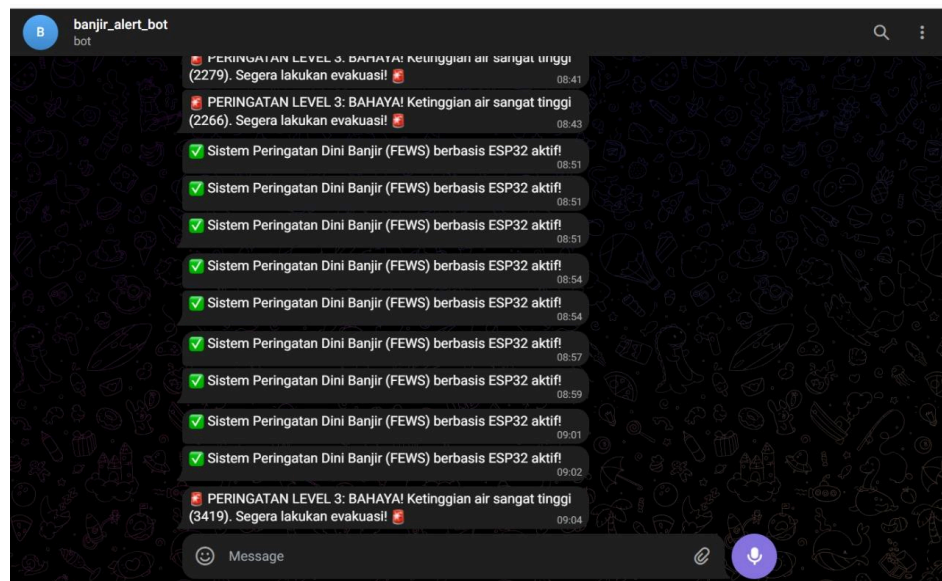
No	Nilai Sensor	Status LED Merah	Status LED Kuning	Status LED Hijau	Status Buzzer	Status Kirim Telegram	Keterangan	Dokumentasi
1.	1.494	OFF	OFF	ON	OFF	Tidak Dikirim	Air di bawah batas aman (AMAN)	 Gambar 3.
2.	1.846	OFF	ON	OFF	OFF	Tidak Dikirim	Air mulai naik, waspada (WASPADA)	 Gambar 4.
3.	1.910	OFF	ON	OFF	ON	Tidak Dikirim	Air mencapai batas bahaya (AWAS)	 Gambar 5.
4.	2.279	ON	OFF	OFF	ON	Dikirim	Air masih tinggi (BAHAYA)	 Gambar 6.

5.	1.703	OFF	ON	OFF	OFF	Tidak Dikirim	Air mulai turun (WASPADA)	
6.	1.505	OFF	OFF	ON	OFF	Tidak Dikirim	Air kembali normal (AMAN)	

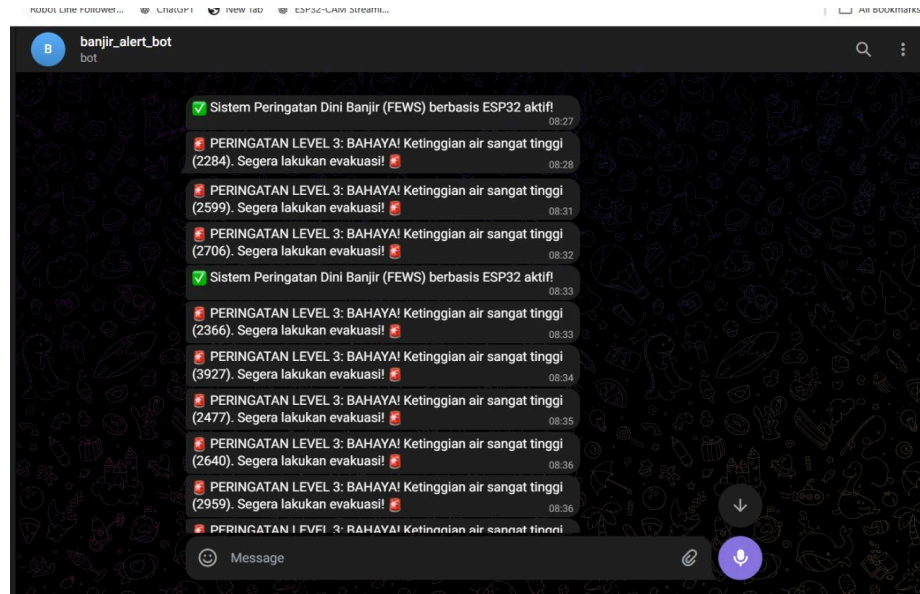
Gambar 7.

Gambar 8.

B. Hasil Pengujian Via Telegram



Gambar 9. Sistem Telah Terhubung ke Telegram Bot



Gambar 10. Sinyal Bahaya Telah Terkirim dari Sistem ke Telegram Bot

C. Analisis

Pengujian dilakukan untuk mengevaluasi kinerja sistem peringatan dini banjir berbasis ESP32 dan Telegram dalam merespons perubahan tinggi muka air. Berdasarkan enam skenario yang diuji, sistem menunjukkan kemampuan yang baik dalam mendeteksi kondisi air dan memberikan peringatan sesuai tingkat bahayanya. Pada kondisi pertama dan keenam, saat nilai sensor menunjukkan angka 1.494 dan 1.505, sistem mendeteksi bahwa air berada di bawah ambang batas aman. Hal ini ditunjukkan dengan nyala LED hijau sementara LED lainnya, buzzer, dan notifikasi Telegram tidak aktif. Kondisi ini mengindikasikan bahwa sistem bekerja dengan baik dalam keadaan normal.

Pada kondisi kedua dan kelima, nilai sensor berada pada kisaran 1.846 dan 1.703, yang menunjukkan bahwa permukaan air mulai naik atau mulai turun. Sistem merespons dengan menyalakan LED kuning sebagai penanda kondisi “WASPADA”, sedangkan LED lainnya, buzzer, dan pengiriman pesan Telegram tetap tidak aktif. Ini menunjukkan bahwa sistem mampu mendeteksi perubahan ketinggian air yang belum mencapai level bahaya namun tetap memberi peringatan visual.

Ketika air mencapai nilai sensor 1.910, sistem masuk ke dalam kondisi “AWAS”. Pada kondisi ini, LED merah dan buzzer aktif, namun sistem belum mengirim pesan melalui Telegram. Hal ini menunjukkan bahwa sistem telah memiliki ambang khusus untuk memisahkan kondisi “AWAS” dari kondisi “BAHAYA”. Pada nilai sensor 2.279, sistem mendeteksi kondisi “BAHAYA”. LED merah dan buzzer kembali aktif, dan yang membedakan adalah sistem juga mengirimkan notifikasi melalui Telegram sebagai bentuk peringatan dini

jarak jauh. Ini membuktikan bahwa fitur komunikasi darurat bekerja dengan baik dan sesuai skenario.

Secara keseluruhan, sistem dapat membedakan tiga kondisi utama, yaitu AMAN, WASPADA, dan BAHAYA, dengan akurasi dan respons yang tepat. Setiap transisi antar kondisi ditandai dengan aktivasi komponen output tertentu (LED, buzzer, dan Telegram) sesuai dengan tingkat keparahan. Dokumentasi gambar yang mendampingi setiap kondisi juga memperkuat bukti keberhasilan pengujian secara nyata dan terukur. Dengan demikian, sistem ini dinilai berfungsi secara optimal dalam memberikan peringatan dini terhadap potensi banjir.

BAB V

Kesimpulan

Dari hasil analisis dan data pada tabel pengujian dapat diambil kesimpulan bahwa sistem Peringatan Dini Banjir berbasis ESP32 dan Telegram ini dapat berfungsi secara efektif. Sistem dapat membedakan berbagai kondisi ketinggian air, mulai dari level AMAN dengan nilai sensor sekitar 1.494 dan 1.505 yang ditandai dengan LED Hijau, meningkat ke WASPADA pada nilai sensor 1.846 (air naik) dan 1.703 (air turun) yang ditandai dengan LED Kuning, berlanjut ke kondisi AWAS pada nilai sensor 1.910 yang ditandai dengan LED Merah dan Buzzer, lalu mencapai puncaknya pada kondisi BAHAYA dengan nilai sensor 2.279 yang ditandai dengan LED Merah, Buzzer, dan pengiriman notifikasi Telegram. Respons peringatan yang responsif dan bertingkat ini, didukung oleh setiap komponen output termasuk keberhasilan pengiriman notifikasi jarak jauh melalui Telegram pada level bahaya, yang artinya sistem ini bisa diandalkan sebagai solusi peringatan dini terhadap potensi banjir.

Lampiran

1. Link Video Demo

<https://drive.google.com/drive/folders/1IU5hvrwvIUecAbFnjexlUdaE6VQ95zEs>

2. Source Code

```
#include <Wire.h> // Diperlukan untuk komunikasi I2C
#include <LiquidCrystal_I2C.h> // Library untuk LCD I2C
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

// --- Pin untuk Komunikasi I2C LCD ---
// Pin I2C default pada ESP32 biasanya:
#define I2C_SDA_PIN 21 // Pin SDA untuk LCD I2C
#define I2C_SCL_PIN 22 // Pin SCL untuk LCD I2C

// Konfigurasi LCD I2C
// Ganti alamat 0x27 jika LCD Anda menggunakan alamat lain (misalnya 0x3F)
// Ganti 16, 2 jika ukuran LCD Anda berbeda (misalnya 20, 4 untuk 20x4)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Konfigurasi WiFi dan Telegram Bot
const char* ssid = "Voltase_Homestay"; // Ganti dengan nama WiFi Anda
const char* password = "asramalistrik"; // Ganti dengan password WiFi Anda
const char* botToken =
"8062922768:AAHWFatUwufqVur5pEYmTWmODK03-g28KF0"; // Ganti dengan token Bot Telegram Anda
const char* chat_id = "6200742914"; // Ganti dengan Chat ID Telegram Anda

WiFiClientSecure client;
UniversalTelegramBot bot(botToken, client);

// Pin Sensor dan Aktuator LED diskrit
#define SENSOR_PIN 34 // Pin analog untuk sensor water level
#define LED_HIJAU 14 // Pin LED Hijau (Kondisi Aman/Normal - Berkelip)
#define LED_KUNING 12 // Pin LED Kuning (Kondisi Waspada & Awas)
```

```

#define LED_MERAH 13      // Pin LED Merah (Kondisi Bahaya)
#define BUZZER 27          // Pin Buzzer

// Ambang Batas Ketinggian Air (Sesuaikan dengan kalibrasi sensor
Anda)
const int THRESHOLD_NORMAL_MAX = 1600;
const int THRESHOLD_SIAGA_MAX = 1850;
const int THRESHOLD_AWAS_MAX = 2200;

// Variabel untuk status dan notifikasi
int lastLevelSent = -1; // Untuk logika Telegram
unsigned long lastSendTime = 0;
const unsigned long telegramInterval = 30000;

// Variabel untuk LED Hijau Berkelip
unsigned long greenLedPreviousMillis = 0;
const long greenLedBlinkInterval = 500;
bool greenLedState = LOW;

// Variabel untuk melacak status terakhir yang ditampilkan di LCD
agar tidak sering update
int lastDisplayedLevelOnLCD = -1;

void setup() {
    Serial.begin(115200);

    // Inisialisasi I2C (untuk LCD)
    // Jika Anda ingin menggunakan pin I2C non-default, Anda bisa
    memanggil Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    // Namun, untuk pin default ESP32 (GPIO 21 untuk SDA, GPIO 22 untuk
    SCL), Wire.begin() saja sudah cukup.
    Wire.begin();

    // Inisialisasi LCD
    lcd.init();          // Inisialisasi LCD
    lcd.backlight();     // Nyalakan backlight LCD
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Flood Early");
    lcd.setCursor(0, 1);
    lcd.print("Warning Sistem"); // Ganti dengan versi atau nama sistem
    Anda

```

```

delay(3000);

//Slide ke 2
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Kelas V2");
lcd.setCursor(0, 1);
lcd.print("Kelompok 3"); // Ganti dengan versi atau nama sistem
Anda
delay(3000);

pinMode(LED_HIJAU, OUTPUT);
pinMode(LED_KUNING, OUTPUT);
pinMode(LED_MERAH, OUTPUT);
pinMode(BUZZER, OUTPUT);

digitalWrite(LED_HIJAU, LOW); // Mulai dengan LED hijau mati
digitalWrite(LED_KUNING, LOW);
digitalWrite(LED_MERAH, LOW);
digitalWrite(BUZZER, LOW);

// Koneksi ke WiFi
Serial.print("Menghubungkan ke WiFi: ");
Serial.println(ssid);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("WiFi Connect...");

WiFi.begin(ssid, password);
int wifi_dots = 0;
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
    lcd.setCursor(wifi_dots, 1); // Menampilkan titik-titik di baris
kedua LCD
    lcd.print(".");
    wifi_dots++;
    if (wifi_dots > 15) { // Jika sudah 16 titik, reset titik di LCD
        lcd.setCursor(0,1);
        lcd.print("                "); // Bersihkan baris kedua
        wifi_dots = 0;
    }
}

```

```

    // Anda bisa menambahkan timeout di sini jika diinginkan,
    // agar tidak terjebak selamanya jika WiFi tidak tersedia
}
Serial.println("\nWiFi tersambung");
Serial.print("Alamat IP: ");
Serial.println(WiFi.localIP());

lcd.clear();
lcd.setCursor(0,0);
lcd.print("WiFi Terhubung!");
lcd.setCursor(0,1);
lcd.print(WiFi.localIP());
delay(2500);

client.setInsecure();
bot.sendMessage(chat_id, "✅ Sistem Peringatan Dini Banjir (FEWS)
berbasis ESP32 aktif!", "");

lcd.clear();
lcd.setCursor(0,0);
lcd.print("Sistem Siap!");
lcd.setCursor(0,1);
lcd.print("Monitoring Air..");
}

// Fungsi untuk menampilkan pesan ke LCD dan padding untuk
membersihkan sisa baris
void printToLCD(int col, int row, String text, int lcdWidth = 16) {
    lcd.setCursor(col, row);
    lcd.print(text);
    // Beri spasi untuk menghapus sisa karakter di baris itu jika
text lebih pendek dari lcdWidth
    for (int i = text.length(); i < lcdWidth; i++) {
        lcd.print(" ");
    }
}

void loop() {
    int sensorValue = analogRead(SENSOR_PIN);
    // Komentari Serial.print di bawah jika tidak ingin terlalu banyak
output di Serial Monitor

```

```

// Serial.print("Nilai Sensor ADC: ");
// Serial.println(sensorValue);

// Reset LED Kuning, Merah, dan Buzzer di awal loop. LED Hijau
dikelola terpisah.
digitalWrite(LED_KUNING, LOW);
digitalWrite(LED_MERAH, LOW);
digitalWrite(BUZZER, LOW);

int currentSystemLevel = 0;
String telegramMessage = "";
String lcdStatusLine1 = "";
String lcdStatusLine2Prefix = ""; // Teks sebelum nilai sensor di
LCD

// 1. Penentuan level kondisi berdasarkan nilai sensor
if (sensorValue < THRESHOLD_NORMAL_MAX) {
    currentSystemLevel = 0; // Normal/Aman
    lcdStatusLine1 = "NORMAL";
    lcdStatusLine2Prefix = "Air Aman";
} else if (sensorValue < THRESHOLD_SIAGA_MAX) {
    currentSystemLevel = 1; // Waspada/Siaga
    lcdStatusLine1 = "WASPADA";
    lcdStatusLine2Prefix = "Air Naik";
} else if (sensorValue < THRESHOLD_AWAS_MAX) {
    currentSystemLevel = 2; // Awas
    lcdStatusLine1 = "AWAS!";
    lcdStatusLine2Prefix = "Air Tinggi";
} else {
    currentSystemLevel = 3; // Bahaya
    lcdStatusLine1 = "BAHAYA!!!";
    lcdStatusLine2Prefix = "EVAKUASI!";
}

// Hanya update LCD jika status level berubah atau ini adalah loop
pertama
if (currentSystemLevel != lastDisplayedLevelOnLCD ||
lastDisplayedLevelOnLCD == -1) {
    printToLCD(0, 0, "Status: " + lcdStatusLine1);

    // Format baris kedua untuk menyertakan nilai sensor
    char lcdLine2Buffer[17]; // 16 karakter + null terminator

```

```

    snprintf lcdLine2Buffer, sizeof lcdLine2Buffer, "%s (%d)",
lcdStatusLine2Prefix.c_str(), sensorValue);
    printToLCD(0, 1, lcdLine2Buffer);

    lastDisplayedLevelOnLCD = currentSystemLevel; // Update status
    terakhir yang ditampilkan
    // Baris Serial.print di bawah ini bisa diaktifkan untuk debug
    jika LCD tidak menampilkan sesuai harapan
    // Serial.print("LCD Updated - Status: " + lcdStatusLine1 + ",
    Detail: " + lcdStatusLine2Prefix + " (" + String(sensorValue) +
    ")\n");
}

// 2. Logika Aktuator berdasarkan currentSystemLevel

// LED Hijau: Berkelip jika Normal (Level 0), Mati jika tidak.
if (currentSystemLevel == 0) {
    unsigned long currentMillis = millis();
    if (currentMillis - greenLedPreviousMillis >=
greenLedBlinkInterval) {
        greenLedPreviousMillis = currentMillis;
        greenLedState = !greenLedState;
        digitalWrite(LED_HIJAU, greenLedState);
    }
    // Komentari Serial.println jika tidak ingin terlalu banyak
    output status di Serial Monitor
    // Serial.println("Status: NORMAL (Air aman terkendali) - LED
    Hijau Berkelip");
} else {
    digitalWrite(LED_HIJAU, LOW); // Pastikan LED Hijau mati jika
    bukan level Normal
}

// LED Kuning: Menyala untuk Waspada (Level 1) dan Awas (Level 2).
// Buzzer: Menyala untuk Awas (Level 2).
if (currentSystemLevel == 1) { // Waspada
    digitalWrite(LED_KUNING, HIGH);
    // Serial.println("Status: WASPADA / SIAGA (Peringatan Level 1 -
    Ketinggian air meningkat)");
} else if (currentSystemLevel == 2) { // Awas
    digitalWrite(LED_KUNING, HIGH);

```

```

        digitalWrite(BUZZER, HIGH);
        // Serial.println("Status: AWAS (Peringatan Level 2 - Ketinggian
air tinggi!)");
    }

    // LED Merah dan Buzzer: Menyala untuk Bahaya (Level 3).
    // Notifikasi Telegram juga dikirim untuk Level 3.
    if (currentSystemLevel == 3) { // Bahaya
        digitalWrite(LED_MERAH, HIGH);
        digitalWrite(BUZZER, HIGH);
        // Menambahkan nilai sensor ke pesan Telegram
        telegramMessage = "🚨 PERINGATAN LEVEL 3: BAHAYA! Ketinggian air
sangat tinggi (" + String(sensorValue) + "). Segera lakukan evakuasi!
🚨";
        // Serial.println("Status: BAHAYA (Peringatan Level 3 -
Ketinggian air SANGAT TINGGI, EVAKUASI!)");

        // Kirim notifikasi Telegram HANYA untuk kondisi BAHAYA (Level 3)
        // jika terjadi perubahan dari level lain ke Level 3, serta
interval terpenuhi
        if (lastLevelSent != 3) {
            if (millis() - lastSendTime > telegramInterval) {
                bot.sendMessage(chat_id, telegramMessage, "");
                Serial.println("Notifikasi Telegram BAHAYA terkirim: " +
telegramMessage);
                lastSendTime = millis(); // Perbarui waktu pengiriman
terakhir
            } else {
                Serial.println("Status BAHAYA terdeteksi, namun interval
pengiriman Telegram belum terpenuhi.");
            }
        }
    }

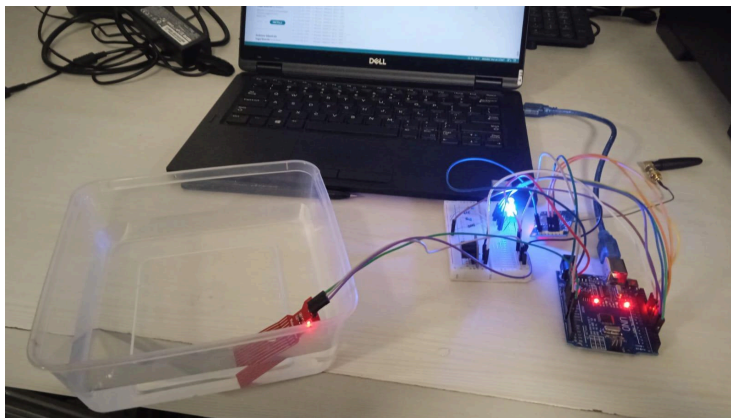
    // 3. Perbarui status level terakhir yang terproses untuk logika
Telegram
    if (currentSystemLevel != lastLevelSent) {
        // Serial.print("Perubahan Level (untuk Telegram): Dari "); //
Bisa diaktifkan untuk debug
        // Serial.print(lastLevelSent);
        // Serial.print(" ke ");
        // Serial.println(currentSystemLevel);
    }

```

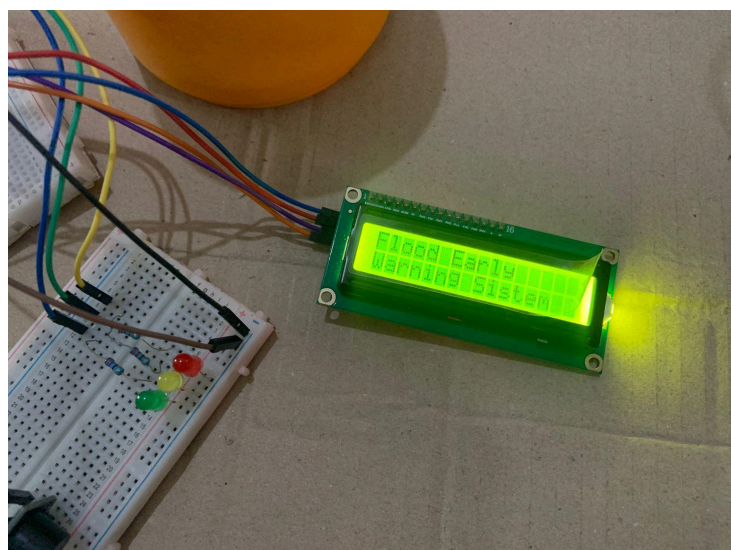
```
    lastLevelSent = currentSystemLevel;
}

delay(100); // Delay loop utama agar kedipan LED lebih responsif
dan pembacaan sensor lebih sering.
}
```

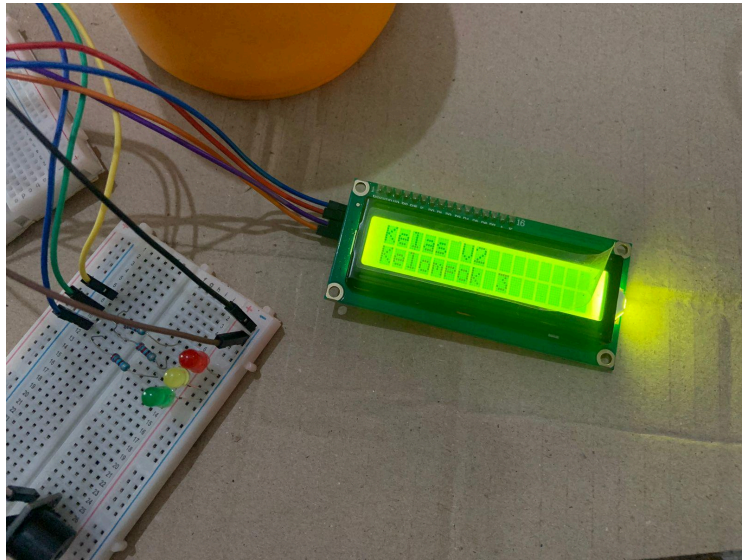
3. Foto Dokumentasi



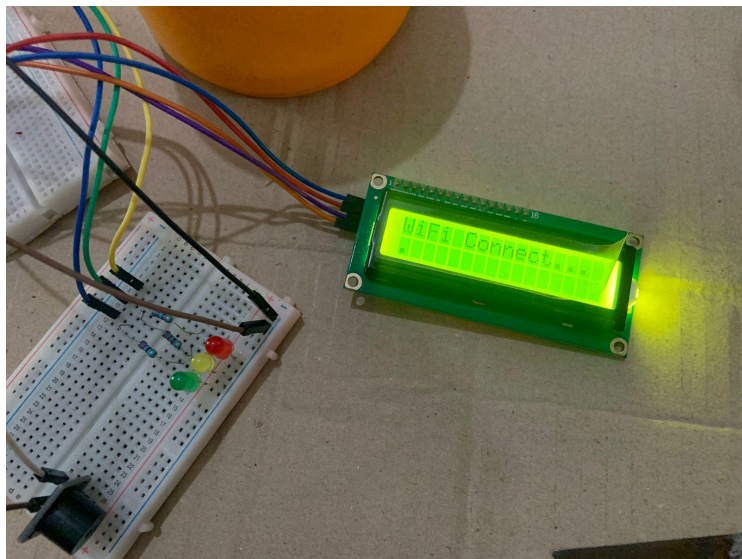
Gambar 11. Proses Pengujian Sistem Pendeteksi Banjir



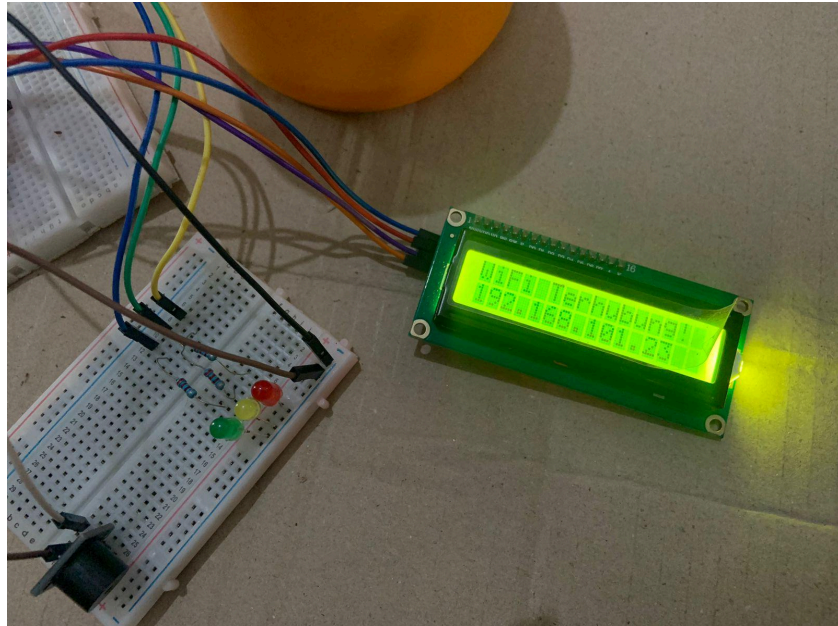
Gambar 12. Tampilan awal LCD (Judul Project)



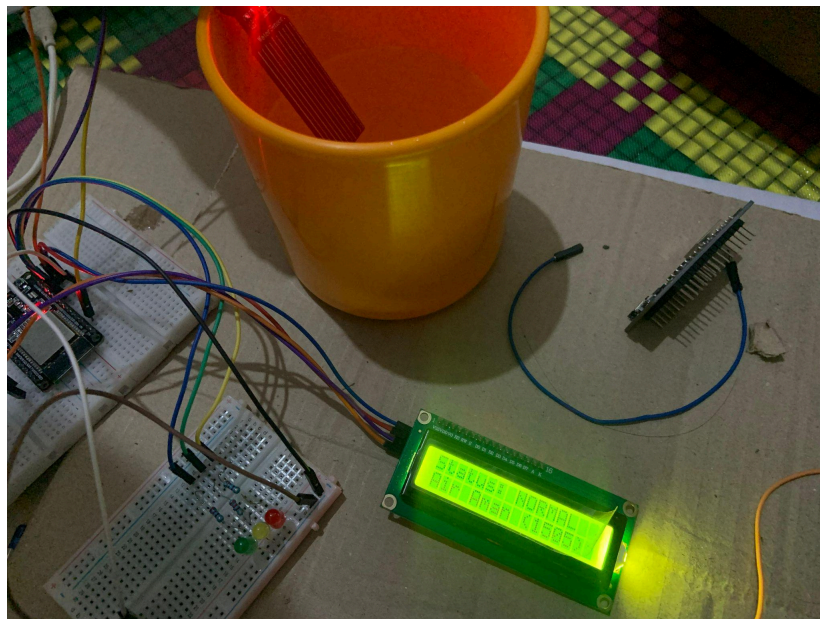
Gambar 13. Tampilan Layar Selanjutnya Mengenai Nama Kelas dan Kelompok



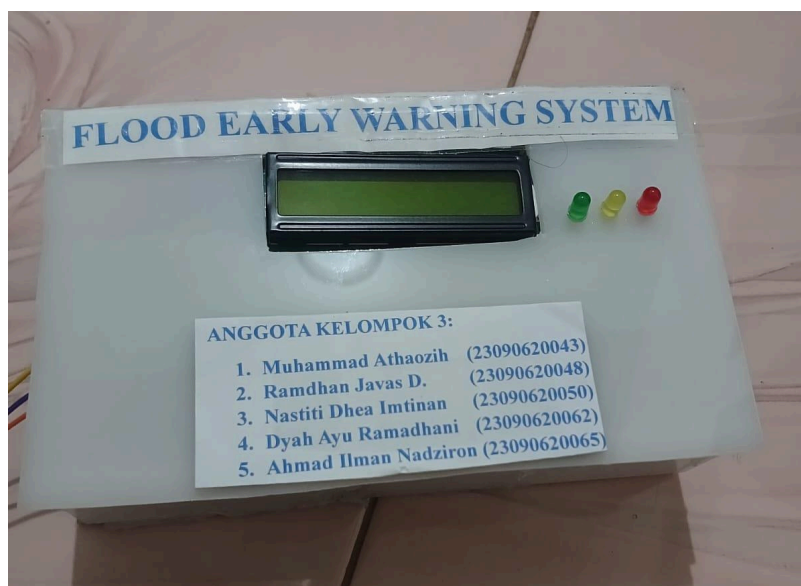
Gambar 14. Tampilan Connect ke wifi



Gambar 15. Tampilan LCD bahwa Telah Terhubung ke WIFI dengan Menampilkan IP



Gambar 16. Tampilan Ketika Normal dengan keadaan aman



Gambar 17. Tampilan Hardware Sistem Pendeteksi Banjir



Gambar 18. Sensor Water Level yang Digunakan untuk Sistem Pendeteksi Banjir